# Modular Arithmetic and GCD

$\%$ → gives remainder

- when no. too large to fit into standard data types like (int) or (long) we use these properties to keep the results within a manageable range:
  - Addition : $(a+b)\%m = (a\%m + b\%m)\%m$
  - Multiplication: $(a*b)\%m = (a\%m * b\%m)\%m$
  - Subtraction: $(a-b)\%m = (a\%m - b\%m + m)\%m$
  - Power : $(a^b)\%m = (a\%m)^b\%m$

## Negative no. in modulo

Java, C++ → $-3\%5 \rightarrow -3$, so to get +ve remainder divisor is added

Python → $-3\%5 \rightarrow 2$

---

① Apply Modulo on # Fast Power

ex: $6^{200}\%15$ $(a^b\%m)$

$a \in (1, 10^9)$
$b \in (1, 10^9)$
$m \in (2, 10^9)$

Pseudo

```
long fastpow (A, B, M) {
    if (B==0) return 1%M
    a2 = fastpow (A, B/2, M)
    if (B%2 == 0) {
        return (a2 * a2) %M      // result
    }
    else { return (A * a2%M * a2)%M }   // or result * a2
```

see assignment 1 res

---

② # GCD → Greatest common divisor

Properties:

1) $\gcd(a, b) = \gcd(b, a)$
2) $\gcd(a, 0) = a$
3) $\gcd(a, b-a) = \gcd(a, b)$ $[b >= a]$
4) $\gcd(a, b-a) = \gcd(a, b-a-a) = \gcd(a, b-a-a..)$
   $\Rightarrow \gcd(a, b\%a)$

// Eucledean Algorithm: $\gcd(a,b) = \gcd(b, a\%b)$

## gcd using property ④:

T.C = $O(\log(\min(a,b)))$
S.C = $O(\log(\min(a,b)))$

```
public int gcd (int a, int b){
        if (a==0) return b;
        return gcd (b%a, a);
}
```

→ automatically sets b as smaller

---

③ Q.# Given an Integer array A[] of size N, delete one element such that the GCD of the remaining array is maximum. Return this maximum G.C.D.

ex: A = [12, 15, 18]

Delete 12 → GCD (15, 18) = 3
Delete 15 → GCD (12, 18) = 6 ☑
Delete 18 → GCD (12, 15) = 3

→ Max. GCD = 6

Sol.

```
public int maximizeGCD (int[]A){
    int n = A.length;
    int[] P = new int [n+1];
    int[] S = new int [n+1];

    // Base cases.
    P[0] = 0        // G.C.D of empty ~~suffix~~ prefix
    S[n] = 0        // G.C.D of empty suffix

    // Build Prefix G.C.D
    for (int i=0 ; i<n ; i++){
        P[i+1] = gcd (P[i], A[i]);
    }

    // Build suffix G.C.D
    for (int i=n-1 ; i>=0 ; i--){
        S[i] = gcd (S[i+1], A[i]);
    }

    int maxGCD = 0;
    // Remove each index
    for (int i=0 ; i<n ; i++) {
        int current GCD = gcd (P[i], S[i+1]);
        maxGCD = Math.max (maxGCD, current GCD);
    }
    return maxGCD;
}
```